

Articulated Object Understanding from a Single Video Sequence

Arslan Artykov Clémentin Boittiaux Vincent Lepetit
LIGM, École des Ponts et Chaussees, IP Paris, CNRS, France

arslan.artykov@enpc.fr, clementin.boittiaux@enpc.fr, vincent.lepetit@enpc.fr

Abstract

This paper introduces a novel method for estimating the structure and joint parameters of articulated objects from a single casual video, captured by a potentially moving camera. Unlike previous works that rely on multiple static views or a priori knowledge of the object category, our approach leverages 2D point tracking and depth map prediction to generate 3D trajectories of points on the object. By analyzing these trajectories, we generate and evaluate hypotheses about joint parameters, selecting the best combination using the Bayesian Information Criterion (BIC) to avoid overfitting. We then optimize a dense 3D model of the object using Gaussian Splatting, guided by the selected joint hypotheses. Our method accurately recovers the geometry, segmentation into parts, joint parameters, and motion of each part, enabling the rendering of the object from new viewpoints and under new articulation states. Extensive evaluations on several datasets demonstrate the effectiveness of our approach.

1. Introduction

Recently, a significant number of works have focused on recovering the geometry and structure of articulated objects. Such objects include tools, furniture, and various equipment and are ubiquitous in our daily environment. The number of joints, their nature, their locations on the object, etc. vary significantly across objects of different categories, and this makes understanding them extremely challenging.

Some methods assume a priori knowledge about the object category [5, 7, 15, 15, 21]. To avoid being restricted to known categories, one current trend is to assume that multiple views of the target object in the same static “state” are available. This is illustrated in Figure 1: The geometry of the object can then be reconstructed very accurately. Many methods also assume that a second set of views of the object in a second state, after the articulated parts moved, is available [18, 35]. They can then estimate very accurately the joint parameters of the parts that moved between the two states. Alternatively, instead of a second set of images in a

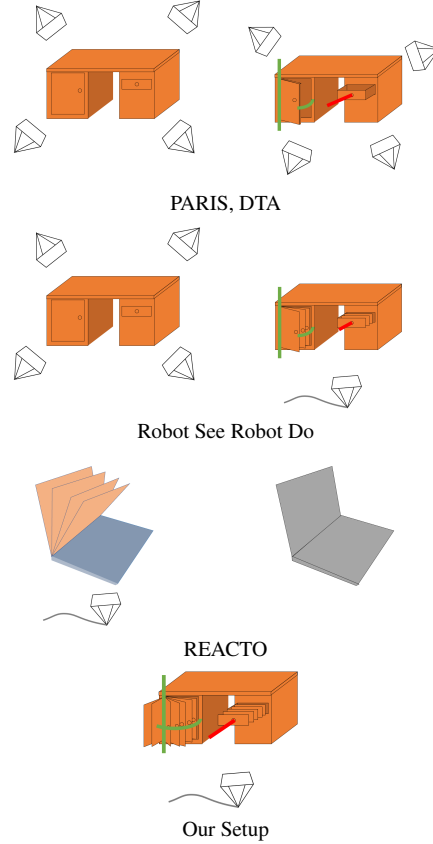


Figure 1. **Comparing the setups of previous methods and ours.** PARIS [18] and DTA [35] assume that two sets of views of the object under two different states are available. Robot See Robot Do [12] needs only the first set of views and recovers the parts and their motions from a video. REACTO [29] only needs a video but reconstructs the object surface only and does not provide the joint parameters of the part. We consider the case when we only have a video as input, but are still able to provide accurate estimate of the joint parameters.

static state, [12] estimates the joint parameters from a video.

However, it is not always possible to capture images while the object is not moving. For example, our ultimate goal is to recover the geometry and joints of large vehicles

such as bulldozers and cranes while they operate and it is not possible to immobilize them. Some methods can reconstruct the shape of an object from a single video captured while the object is moving [29]. But they do not provide estimates of the joints, for example the rotation axis for a revolute motion or the translation axis for a prismatic motion. They also have not been demonstrated on prismatic motions, i.e., translating parts, as their surface deformation representation is not well adapted to such cases.

In this paper, we thus explore how the structure of articulated objects and the parameters of their joints can be estimated from a single casual video captured by a possibly moving camera. We want to estimate automatically the number of parts the object has, the translation and rotation axes of its joints, and by how much translates or rotates each part moves in each frame of the video. Like previous works, we can then render the object from new viewpoints and under new articulation states, but with a single video as input.

To deal with the complexity of recovering an a priori unknown number of parts from a single video, our starting idea is to generate a set of hypotheses about joint parameters. We leverage recent progress on 2D point tracking [8] and depth map prediction [26] to generate 3D trajectories of points lying on the object. Despite the impressive progress in 2D tracking and depth prediction, such trajectories can still be very noisy, as shown in Figure 2. Fortunately, we can observe that an articulated motion is extremely constrained. A prismatic motion is defined entirely by a translation vector and an amount of translation at each time step; a revolute motion is defined by a rotation axis, pivot point, and an angle for each time step. From a single trajectory of a 3D point on a part, it is possible to estimate the motion parameters of this part. We identify good joint hypotheses by looking for joint parameters that explain large subsets of 3D trajectories.

We then want to select which combination of joint hypotheses explain best the input video. We do this by looking at the 3D trajectories and how well they can be explained by each combination of joint hypotheses. We will show that this can be done very efficiently, however this has to be done with care. In the classic problem of fitting a polynomial to a set of points, increasing the polynomial degree results in a better fit but also in undesirable overfitting. In our case, this could result in predicting more parts that the object actually has. We thus turn to model selection theory. More exactly, we rely on the Bayesian Information Criterion (BIC) to select the right combination of joint hypotheses.

Finally, like previous works [37], we obtain a dense 3D model of the object using Gaussian Splatting. More exactly, we optimize on the Gaussians parameters and the joint parameters. The joint hypotheses retained by our selection guide the assignment of the Gaussians to their correct parts. Thank to this initialization, this large optimization problem

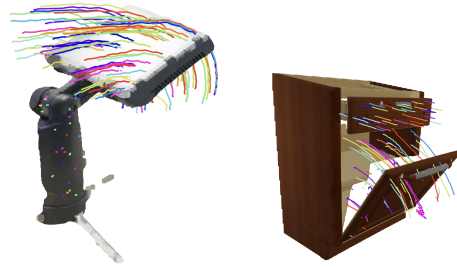


Figure 2. **3D trajectories recovered by 2D point tracking and depth prediction.** We visualize here some of the 3D trajectories we recover from a video together with our reconstruction of the target object. Left: an articulated lamp from a real video of RSRD [12], right: a Storage-41083 object from our Multi-Part Object Dataset. These trajectories capture the articulated motions, however they can be very noisy. We show how to recover robustly and accurately the joint parameters of the objects from them.

converges efficiently towards the correct solution.

The accuracy we obtain in terms of joint parameters is lower than earlier works. However, remember that these methods rely on multiple views of the target object under a static pose. In our case, we consider as input a single video captured when the object and possibly the camera are moving, which makes the problem significantly more challenging. Nevertheless, we manage to recover geometry and joint parameters accurately enough to render the objects novel views and states.

To summarize, our contribution is a novel method able to recover the the geometry of an articulated object, its segmentation into parts, the joint parameters, and the motion of each part from a single video, without any prior knowledge on how the object is structured. We extensively evaluate our method on several datasets to demonstrate its performance.

2. Related work

Articulated object modeling. Articulated object modeling is crucial in computer vision and has huge potential in robotics. While several methods have been proposed to address this problem [2, 3, 5–7, 10, 15, 18, 21, 30, 34, 35], modeling unknown objects remains a challenging problem.

Some work rely heavily on machine learning to recover the structure of articulated objects from known categories. For example, A-SDF [21] which builds upon the success of implicit representation as in DeepSDF [24] in modeling category-level articulation by encoding joint angles within the shape code. ANCSH [15], inspired by the normalized object coordinate space (NOCS) for rigid objects [31], introduces an extended version to model canonical representations of articulated objects. CAPTRA [34] then presents a unified framework that can be applied for tracking task from point cloud sequences. However, as these methods consider

object categories, they cannot generalize to unknown objects such as those from novel categories. Ditto [7] can predict motion and geometry across object categories using two 3D point clouds but still struggles with detailed appearance reconstruction for objects of unknown categories as shown in our experiments.

[23] consider hypotheses on the articulated parts, but in a way very different from ours: It learns to predict movable parts and try interacting physically with them to test these hypotheses. Instead, we predict hypotheses based on motions and validate these hypotheses geometrically.

Instead of relying on learning, [22] relies on 3D geometry, which is captured by multiple video sequences captured while the target object is moving. To simplify this setup, PARIS [18] and DigitalTwin (DTA) [35] do not require multiple sequences, but two sets of frames of the object: The object has to be in the same configuration in all the images of each set, and the methods recover the articulated parts that moved between the two sets of images.

REACTO [29] and Articulate-Anything [14] have been introduced to consider single videos as input. REACTO can recover the surfaces of the target objects, however, it does not provide estimates for the joint parameters. Articulate-Anything can predict joint parameters of single-part objects, but it does not reconstruct object geometry; instead, it retrieves a mesh from an existing database.

Like Articulate-Anything, our method takes a single video as input, but it accurately estimates an arbitrary number of joint parameters and reconstructs the geometry of the object.

Dynamic scene reconstruction using 3D Gaussians.

Many approaches have been proposed to tackle the task of dynamic scene reconstruction using 3D Gaussians [1, 9, 13, 16, 17, 19, 36, 37]. A relevant example is Dynamic 3D Gaussians [19] which targets reconstructing a scene using several cameras for each time step. Another similar approach is 4D Gaussian Splatting [36], which uses much lower storage to achieve real-time dynamic scene rendering. More recently, Deformable-3DG [37] has been proposed to reconstruct scenes and learn Gaussians in canonical space with a deformation field to model dynamic scenes. [33] represents this deformation field as a linear combination of rigid motion.

In our case, we are not interested in recovering general deformations but piecewise rigid deformations that happen on articulated objects. Moreover, we want to recover the joint parameters, such as the rotation axes of hinges or the 3D directions of prismatic motions.

Several recent works also aim at recovering articulated objects and their joint parameters [18, 35]. These works rely on two sets of views, capturing the 3D geometry of the object in two different states with Gaussians and optimizing

on articulated motions that transform the Gaussians from the first state to the second one.

We use Gaussians as well to represent the target object as they are very convenient to capture geometry from images. However, in our case, we have only a video as input. This means that we capture each object joint state from a single point of view. To compensate for this limited information, we rely on 3D trajectories to capture the motions over time and show how we can infer the joint parameters of the objects from these trajectories, in addition to the object geometry.

3. Method

In this section, we first formalize the problem we want to solve by introducing a loss function that depends on the geometry of the object and its articulated parts and on the joint parameters. We then explain how we generate and select hypotheses on the articulated parts and how they move. This gives us a very good initialization, from which we can optimize our loss function and recover geometry and joint parameters.

3.1. Formalization

Like earlier methods, we choose to represent the target object using 3D Gaussians as they are an efficient way to explicitly capture 3D geometry of the object.

We optimize a set $\{G_g^{\text{Can}}\}_g$ of Gaussians. Each G_g^{Can} is a 3D Gaussian defined in a canonical coordinate system, e.g., the coordinates in the first frame of the video. g denotes the index of the Gaussian. As in regular Gaussian Splatting, each Gaussian is parameterized by its center, covariance, and spherical harmonics coefficients for its colors.

We will also optimize the joint parameters M_p^t of each articulated part. p denotes the part index and t the time step. A prismatic motion is parameterised by a translation direction plus an amount of translation for each time step. A revolute motion is parameterised by a rotation axis, pivot point, and an amount of rotation for each time step. The supplementary material gives more details about the representation of the motion.

We use the notation $M_{p_g}^t \circ G_g^{\text{Can}}$ to represent Gaussian G_g^{Can} after applying motion parameterised by M_p^t .

In addition to the standard parameters of the Gaussians, we also assign a vector w_g to each Gaussian, where g is the Gaussian’s index. w_g is a vector of dimension P , where P is the number of parts. Note that we do not assume that P is given, and we also aim to recover it. We use w_g to estimate the probabilities of Gaussian g to belong to each part by taking:

$$P(G_g^{\text{Can}} \text{ belongs to part } p) = \text{softmax}(w_g)_p. \quad (1)$$

We thus want the n -th coordinate of w_g to be large at convergence if Gaussian g belongs to part n , while the other

coordinates should remain small. Then:

$$\text{softmax}(w_g)^\top \cdot [M_1^t \circ G_g^{\text{Can}}, \dots, M_P^t \circ G_g^{\text{Can}}]^\top, \quad (2)$$

with $^\top$ the transpose operator, is the expected trajectory of Gaussian g when t varies.

In principle, we can then optimize the following loss function

$$\sum_t \text{GS} \left(\bigcup_g \text{softmax}(w_g)^\top \cdot [M_1^t \circ G_g^{\text{Can}}, \dots, M_P^t \circ G_g^{\text{Can}}]^\top, I_t \right) \quad (3)$$

over Gaussians $\{G_g^{\text{Can}}\}_g$, joint parameters $\{M_p^t\}_{p,t}$, and vectors $\{w_g\}_g$, to recover the object geometry, its parts, and its joint parameters from video frames I_t . $\text{GS}(G, I_t)$ is the standard Gaussian Splatting function comparing a rendering of set G of Gaussians with image I_t .

This is however a very difficult optimization problem with many parameters of different natures. It is therefore critical to start from a good first estimate of the motions and the segmentation of the Gaussians into parts. We detail our strategy below.

3.2. Generating Articulated Motion Hypotheses

To generate hypotheses on the joints and their parameters, we first extract 3D trajectories of points lying on the object surface by running CoTracker3 [8] from each frame of the sequence. CoTracker3 tracks 2D points in the previous and next frames until they become occluded. We backproject these 2D points using depth maps, either acquired by the camera or predicted using a model—we use UniDepth [26] for real sequences in practice. This yields 3D trajectories $T_i = \{N_i^t\}_{t_b^i \leq t \leq t_e^i}$. Each trajectory has its own time interval $[t_b^i; t_e^i]$, depending on how far the point could be tracked along the input video, starting from frame t_b^i .

As shown in Figure 2, these trajectories capture the part motions but can be significantly noisy. We seek subsets of trajectories that can be explained by articulated motions. Fortunately, an articulated motion is extremely constrained. A prismatic motion is defined entirely by a translation vector and an amount of translation at each time step. A revolute motion is defined by a rotation axis, pivot point, and an angle for each time step. In fact, as shown in Figure 3, it is possible to estimate these joint parameters from a single trajectory of a 3D point.

We thus propose to generate good hypotheses for the articulated motions by randomly selecting one trajectory T_i , randomly pick a type of articulated motion—either prismatic or revolute, and compute the joint parameters for this trajectory. We then check if there are many other trajectories that can also be explained by these joint parameters. When it is the case, we keep the computed joint parameters as a good hypothesis. We remove the trajectories that can

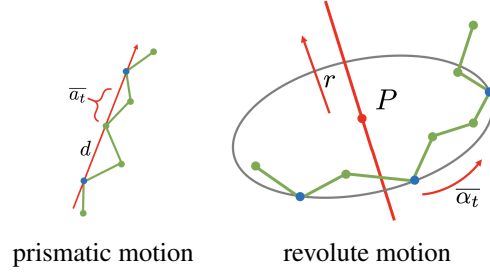


Figure 3. **Generating motion hypotheses.** Left: We generate a prismatic motion by randomly picking a trajectory and two points on this trajectory. We estimate the direction d and the average amount of translation $\bar{a}_t$ between time step. If this trajectory is actually on a prismatic motion, this motion will explain other trajectories and we keep it as a hypothesis. We proceed similarly for revolute motions, which require picking three points.

be explained by this hypothesis, and we iterate until we get the desired number H of hypotheses. In practice, we use $H = 10$.

We give an overview of this generation below for the different types of joints. The supplementary material provides more details and the formulas.

Prismatic joints. If the selected type of motion is “prismatic”, we fit a 3D line to trajectory T_i . This gives us the potential direction d of the motion, but also the amount of translation a_t by projecting point N_i^t on the line. Estimates a_t are however noisy in practice so we consider only their average $\bar{a}_t$. We can then check if this motion hypothesis explains other trajectories. This is done by applying the estimated motion to the origin points of the trajectories to predict its 3D location at time t $\hat{N}_j^t$. If $\hat{N}_j^t$ is close to its observed position N_j^t , this counts as an inlier. If the total number of inliers is sufficiently large, we keep these joint parameters as a good hypothesis.

Revolute joints. If the type of motion is revolute, we fit a 3D circle to this trajectory to obtain the direction of the rotation axis r and a pivot point P . Similarly to the prismatic motion, we can then compute the average amount of rotation $\bar{\alpha}_t$. We also check if this motion hypothesis explains other trajectories, and apply the estimated motion to the origin points of the trajectories to predict its 3D location at time t $\hat{N}_j^t$, this time using the rotational motion. Like for the prismatic motion, if the predicted point location $\hat{N}_j^t$ is close to its observed position N_j^t , this counts as an inlier. If the total number of inliers is sufficiently large, we keep these joint parameters as a good hypothesis.

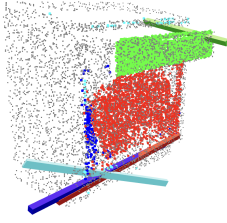


Figure 4. **Generated motion hypotheses** for an object with one prismatic motion and one revolute motion. We visualize points on the inlier trajectories and the motion parameters as the translation or rotation axis. Some hypotheses are correct, some are wrong (such as the cyan one here). Only 4 hypotheses are shown here for clarity, we use 10 hypotheses in our experiments. We propose a method to select the correct ones based on model selection theory.

3.3. Selecting the Motion Hypotheses

The previous stage gives us a set $\mathcal{H}$ of H hypotheses for the articulated motions, with H large enough to avoid missing correct hypotheses in $\mathcal{H}$. As shown in Figure 4, in general, some hypotheses are correct, some are not, and some hypotheses correspond to the same articulated motion. We want to eliminate the incorrect ones but also keep only one hypothesis per object part.

While earlier works either assumed to know the number of parts or relied on heuristics for determining it, we propose to rely on model selection theory. Applied to our problem, a ‘model’ is a subset of the hypotheses and we want to find the combination of hypotheses that explains the object best. More complex models can increase the maximum likelihood—or equivalently decrease the minimum loss function—by adding parameters, but doing so may result in overfitting. The Bayesian Information Criterion (BIC) [28] resolves this problem by introducing a penalty term for the number of parameters in the model:

$$\text{BIC}(\mathcal{C}) = k(\mathcal{C}) \ln(n) + \lambda \mathcal{L}(\mathcal{C}), \quad (4)$$

where $\mathcal{C}$ is a combination of hypotheses in $\mathcal{H}$, $\mathcal{L}(\mathcal{C})$ is a loss function, $k(\mathcal{C})$ is the total number of parameters in $\mathcal{C}$, n is the number of observations, and λ is a hyperparameter, fixed for all experiments. BIC is adapted here because it relies on the assumption that the number of observations n is much larger than the number of parameters k , which is the case for our problem. We seek combination $\hat{\mathcal{C}}$ that minimizes $\text{BIC}(\hat{\mathcal{C}})$.

$k(\mathcal{C})$ is the sum of parameters of hypotheses h in $\mathcal{H}$:

$$k(\mathcal{C}) = \sum_{h \in \mathcal{C}} k(h). \quad (5)$$

$k(h) = 3$ if h is a prismatic motion, because it is parameterized by a direction in 3D space (2 parameters) and an

average amount of translation (1 parameter): Even if we use in practice a 3D vector to represent the direction in 3D space, its number of parameters should be indeed taken to 2 as changing the norm of the vector does not change its direction.

$k(h) = 5$ if h is a revolute motion, because it is parameterized by a 3D line for the rotation axis (4 parameters) and an average amount of rotation (1 parameter): Even if we use in practice a 3D point and a 3D vector to represent the 3D line, this representation is over parameterized. The minimal number of parameters of a 3D line is indeed 4 [4]: One way to see that is to realize that a 3D line can be defined by 2 points each lying on a different fixed 3D plane.

We could use the loss introduced in Eq. (3) for $\mathcal{L}(\mathcal{C})$. However, identifying combination $\hat{\mathcal{C}}$ with the smallest $\text{BIC}(\hat{\mathcal{C}})$ would require optimizing Eq. (3) for every possible combination of hypotheses in $\mathcal{H}$. This would require running 2^H Gaussian Splatting optimizations, which would be prohibitive.

Instead, we use as loss the distance between the observed trajectories $\{N_i^t\}$ and the predicted trajectories $\{\hat{N}_i^t\}$ that were used to generate the hypotheses in $\mathcal{H}$. We show in the supplementary material that $\text{BIC}(\mathcal{C})$ can be computed very efficiently using:

$$\text{BIC}(\mathcal{C}) = \sum_{h \in \mathcal{H}} \mathbb{1}_{h \in \mathcal{C}} B_1(h) + (1 - \mathbb{1}_{h \in \mathcal{C}}) B_2(h), \quad (6)$$

where the $B_1(h)$ and $B_2(h)$ can be precomputed. Computing $\text{BIC}(\mathcal{C})$ for a given combination $\mathcal{C}$ of hypotheses is now a matter of a few additions. To find the combination that minimizes $\text{BIC}(\mathcal{C})$, we simply compute its value for all 2^H possible combinations, and keep the combination that yields the lowest value.

3.4. Initialization and Optimization

We initialize the joint parameters $\{M_p^t\}_{p,t}$ in Eq. (3) to the motion parameters in the hypotheses in $\hat{\mathcal{C}}$. The size of $\hat{\mathcal{C}}$ gives us P , the number of parts.

To initialize the Gaussians, we first select the canonical frame as the one that has the largest number of trajectories. We initialize the Gaussians centers using the points N_i^t of the trajectories in this frame. As in standard Gaussian Splatting, we initialize the covariance matrices to a small isotropic covariance matrix.

Vectors $\{w_g\}_g$ of Gaussians initialized with points from trajectories that are inliers of a motion hypothesis are initialized using the index of this hypothesis. We also use this initialization for the other Gaussians when their 3 nearest neighbors are from inlier trajectories of the same motion hypothesis. The vectors of the remaining Gaussians are drawn from a uniform distribution.

Similarly to [33], we add to Eq. (3) a term that sums the distances between the Gaussians centers and their cor-

		FoldChair	Fridge	Laptop [†]	Two-Part		Blade	All
				Oven [†]	USB			
Axis Ang	Ditto [7]	89.35	89.30*	3.12	0.96	89.77	79.54*	58.67
	PARIS [18]	8.08±13.2	9.15±28.3	0.02±0.0	0.04±0.0	0.13±0.2	15.38±14.9	5.47±9.43
	PARIS* [18]	15.79±29.3	2.93±5.3	0.03±0.0	7.43±23.4	0.71±0.8	41.28±31.4	11.33±15.03
	Articulate-Anything [14]	F	0.00±0.0	0.00±0.0	0.00±0.0	90.00±0.0	0.00±0.0	18.00±0.0
	DTA [35]	0.03±0.0	0.07±0.0	0.06±0.0	0.22±0.0	0.11±0.0	0.27±0.0	0.13±0.0
	Ours	0.34±0.0	1.54±0.0	0.30±0.0	1.46±0.0	0.06±0.0	0.15±0.0	0.64±0.0
Axis Pos	Ditto [7]	3.77	1.02*	0.01	0.13	5.41	-	2.59
	PARIS [18]	0.45±0.9	0.38±1.0	0.00±0.0	0.00±0.0	2.36±3.4	-	0.80±1.33
	PARIS* [18]	0.25±0.5	1.13±2.6	0.00±0.0	0.05±0.2	3.35±3.1	-	1.20±1.6
	Articulate-Anything [14]	F	2.71±0.0	0.06±0.0	0.39±0.0	5.46±0.0	-	1.72±0.0
	DTA [35]	0.01±0.0	0.01±0.0	0.00±0.0	0.01±0.0	0.00±0.0	-	0.01±0.0
	Ours	0.06±0.0	0.24±0.0	0.02±0.0	0.43±0.0	0.05±0.0	-	0.16±0.0
Part Motion	Ditto [7]	99.36	F	5.18	2.09	80.60	F	46.81
	PARIS [18]	131.66±78.9	24.58±57.7	0.03±0.0	0.03±0.0	64.85±84.3	0.34±0.2	36.92±36.85
	PARIS* [18]	127.34±75.0	45.26±58.5	0.03±0.0	9.13±28.8	96.93±67.8	0.36±0.2	46.51±38.3
	Articulate-Anything [14]	-	-	-	-	-	-	-
	DTA [35]	0.16±0.0	0.09±0.0	0.08±0.0	0.11±0.0	0.11±0.0	0.00±0.0	0.09±0.0
	Ours	0.19±0.0	1.01±0.0	2.57±0.0	0.15±0.0	0.54±0.0	0.12±0.0	0.76±0.0
CD-w	Ditto [7]	6.80	2.16	0.31	2.51	2.09	42.04	9.32
	PARIS [18]	1.80±1.2	2.92±0.9	0.30±0.1	11.73±1.1	2.00±0.2	0.60±0.2	3.23±0.62
	PARIS* [18]	4.37±6.4	5.53±4.7	0.26±0.0	3.18±0.3	1.78±0.2	0.58±0.1	2.62±2.0
	Articulate-Anything [14]	-	-	-	-	-	-	-
	DTA [35]	0.27±0.0	0.70±0.0	0.35±0.0	4.18±0.0	1.18±0.0	0.36±0.0	1.17±0.0
	Ours	0.44±0.0	0.78±0.0	0.15±0.0	4.91±0.0	0.30±0.0	0.30±0.0	1.15±0.0

Table 1. **Main metrics on the Two-Part Object dataset** (the complete table can be found in the supplementary material). We report the means and standard deviations of the metrics for PARIS, PARIS*, DTA, and our method, computed over 10 trials. PARIS* [18] is the depth augmented version of PARIS. Objects with [†] belong to the seen categories that Ditto [7] was trained on. Ditto sometimes gives wrong motion type predictions, which are noted with F for joint state and * for joint axis or position. Similarly, Articulate-Anything failed on FoldChair and was noted with F. Note that Blade has only a prismatic joint and no revolute joint, which is why there is no values for the Axis Position. Articulate-Anything does not estimate part motions, nor does it perform geometry reconstruction. Instead, it retrieves object meshes from a pre-existing database. Therefore, we do not report Chamfer Distance for this method.

responding trajectories. Thanks to our initialization and this term based on our trajectories, optimizing Eq. (3) converges very well. It refines the joint parameters, identifies the part each Gaussian belongs to, and recovers the 3D geometry of each part by optimizing the Gaussian parameters.

4. Experiments

In this section, we compare our results to Ditto [7], PARIS [18], DTA [35], CSG-reg [35], 3Dseg-reg [35], PARIS* [18], and PARIS*-m [18]. These methods assume that two sets of views of the object under two different states are available. We consider the case when we only have a video as input, but are still able to provide accurate estimate of the joint parameters. The most comparable recent method is Articulate-Anything [14], which takes a video as input but handles only single-part objects and lacks 3D reconstruction. For fairness, we include it in our comparisons.

Ditto is a deep learning approach that estimates part-level meshes and joint parameters of articulated objects using multi-view point clouds from two distinct joint states as input. It assumes that only a single part is in motion between the two states. PARIS estimates the joint model of a two-part articulated object, including its geometry and appearance. DTA optimizes the joint parameters and geometry of an object using a neural rendering-based approach. PARIS* and PARIS*-m are versions of PARIS that are enhanced with depth information and adapted for multi-part

objects, respectively. CSG-reg and 3Dseg-reg are additional baseline methods introduced by DTA, which reconstruct the scene at each state through TSDF fusion.

4.1. Datasets and Implementation Details

We consider a dataset made of objects with two parts only, as the original PARIS considers such objects only, and a dataset made of objects with more parts.

Synthetic Two-Part Object Dataset. We created a video dataset featuring two-part articulated objects sampled from PartNet-Mobility [20]. To ensure a fair evaluation, we selected the same objects as in PARIS [18]. Each video includes rendered RGB images, depth maps, and object masks, capturing the object’s articulation.

Synthetic Multi-Part Object Dataset. We constructed a dataset of five multi-part articulated objects sampled from PartNet-Mobility [20], two of which were used in DTA [35]. Each object consists of one static part and multiple dynamic parts. Following the approach used for two-part objects, we rendered RGB images, depth maps, and object masks while articulating the objects.

Real-World Scene Dataset. We collected a custom dataset consisting of three articulated objects: a chair, a storage cabinet, and a laptop. In the laptop sequence, the

		Multi-Part					
		Storage-41083	Storage-44781	Fridge-10489	Storage-47254	Box-102377	All
Axis Ang 0	PARIS*-m [18]	-	-	34.52	43.26	-	38.89
	DTA [35]	36.29	6.96	8.47	0.17	3.03	10.98
	Ours	0.36	0.42	2.72	2.05	2.25	1.56
Axis Ang 1	PARIS*-m [18]	-	-	15.91	26.18	-	21.04
	DTA [35]	5.50	83.76	8.47	0.45	38.74	27.38
	Ours	0.74	1.56	0.84	1.09	8.32	2.51
Axis Ang 2	PARIS*-m [18]	-	-	-	-	-	-
	DTA [35]	-	4.81	-	-	-	4.81
	Ours	-	0.75	-	-	-	0.75
Axis Pos 0	PARIS*-m [18]	-	-	3.60	10.42	-	7.01
	DTA [35]	6.22	3.45	5.72	0.04	8.57	4.80
	Ours	0.19	0.31	0.37	0.15	0.12	0.23
Axis Pos 1	PARIS*-m [18]	-	-	1.63	-	-	1.63
	DTA [35]	-	4.84	5.72	-	-	5.28
	Ours	-	1.21	0.00	-	-	1.21
Part Motion 0	PARIS*-m [18]	-	-	86.21	79.84	-	83.03
	DTA [35]	83.36	59.59	7.24	0.12	65.91	43.24
	Ours	0.32	1.47	3.58	0.20	5.29	2.17
Part Motion 1	PARIS*-m [18]	-	-	105.86	0.64	-	53.25
	DTA [35]	0.05	93.83	40.51	0.00	0.73	27.02
	Ours	0.08	2.58	0.11	0.06	0.06	0.58
Part Motion 2	PARIS*-m [18]	-	-	-	-	-	-
	DTA [35]	-	0.05	-	-	-	0.05
	Ours	-	0.07	-	-	-	0.07
CD-w	PARIS*-m [18]	-	-	15.00	8.66	-	11.83
	DTA [35]	3.57	3.18	0.90	0.98	3.85	2.50
	Ours	1.19	0.93	0.50	0.57	1.32	0.90

Table 2. **Results on the Multi-Part Object dataset.** We report here the main metrics; the complete table is given in the supplementary material. PARIS*-m [18] is the depth-augmented, multi-part-adapted version of PARIS. Since the code of PARIS*-m is not open-sourced, we show the results from DTA [35] for Fridge-10489 and Storage-47254. Note that Storage-44781 has 3 joints and the other objects have 2 joints. The first joints of Storage-41083, Storage-47254, and Box-102377 are prismatic, so there is no Axis Pos. for them. Our method outperforms the other methods on all metrics for the multi-part object dataset.

Table 3. **Ablation experiments on model selection.** Lower is better on all metrics.

Method	Storage-47254						Storage-41083					
	Axis Ang	Axis Pos	Part Motion	CD-s	CD-m	CD-w	Axis Ang	Axis Pos	Part Motion	CD-s	CD-m	CD-w
Proposed	1.57	0.15	0.13	0.46	0.13	0.57	0.55	0.19	0.20	1.63	0.79	1.19
2 nd Best BIC	3.13	0.27	0.68	0.47	0.15	0.59	27.56	0.08	0.65	1.24	4.53	2.07
using the number of inliers	3.66	0.41	0.45	0.52	0.14	0.58	1.65	0.21	0.35	1.12	1.78	1.32

camera undergoes dynamic motion, whereas in the chair and storage sequences, the camera remains static. To obtain camera poses and depth maps, we used VGGT [32], and used SAM2 [27] to generate object masks. Detailed qualitative results are provided in the supplementary material.

Implementation details. We implemented our framework using PyTorch [25] and used a single GPU V100 16GB for all our experiments. We use the differentiable Gaussian rasterization from the original Gaussian Splatting [11]. Our non-optimized implementation takes an average of 30 minutes per object, similar to the DTA method.

4.2. Metrics

Following PARIS and DTA, we assessed joint parameter estimation using Axis Ang Error ($^{\circ}$), which measures the angular error of the estimated joint axes for both prismatic and revolute joints. For revolute joints, we also computed Axis Pos Error (0.1m), defined as the minimal distance between the predicted and ground-truth pivot points. Additionally,

we evaluated the predicted part motion between consecutive timesteps using Part Motion Error ($^{\circ}$ or m), which quantifies the geodesic distance error for revolute joints and the Euclidean error for prismatic joints. For the geometry evaluation, we compute the Chamfer Distance (in mm) between two point sets sampled from the estimated and ground-truth meshes. Following DTA, we also report the shape errors for static, whole, and dynamic parts separately.

4.3. Two-Part Object Results

As shown in Table 1 and Figure 5, our method achieves results competitive with DTA and Articulate-Anything, while the results reported for DTA require 10 trials [35]. Since our method involves some randomness (we generate hypotheses by randomly picking trajectories and points on these trajectories), we report the standard deviations for our method as well, however they remain extremely small. Moreover, DTA relies on multi-view images at each articulation state, whereas our method can estimate articulation parameters with similar accuracy from casual videos. In terms of mesh reconstruction accuracy, our method outper-

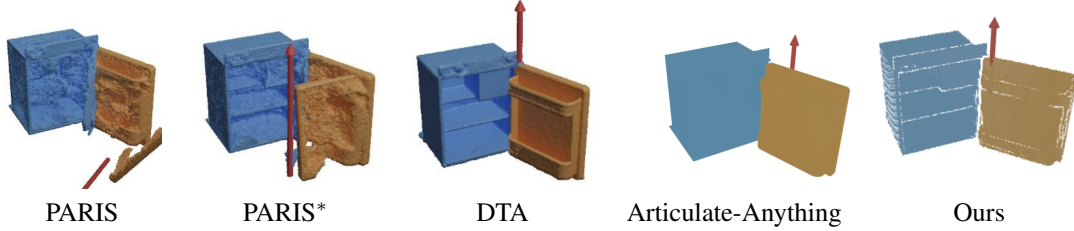


Figure 5. **Visualization of the results on the Fridge object from the two-part object dataset.** We show joint estimation, mesh reconstruction and part segmentation results. Our results are as accurate as DTA while we use a single video. Note that Articulate-Anything retrieves meshes from a pre-existing database. Therefore, the focus here is on evaluating its joint prediction performance.

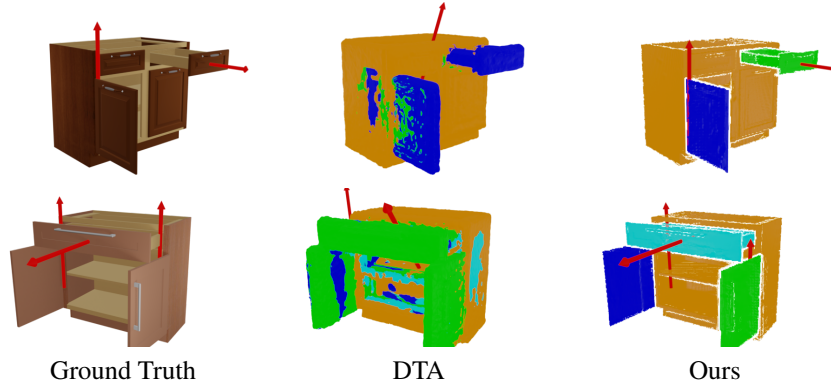


Figure 6. **Qualitative results on the Storage-41083 and Storage-44781 objects from the multi-part object dataset.** We show joint estimation, mesh reconstruction and part segmentation results. Refer to the supplementary material for the rest of the objects and the video.

forms or is on par with all other baselines. Additionally, Articulate-Anything does not reconstruct 3D shape of objects, however, our method outputs both geometry reconstruction and joint parameter estimations.

4.4. Multi-Part Object Results

Table 2 presents the evaluation of our method on multi-part objects. Since, Articulate-Anything can not predict joint parameters of multi-part objects, we compared our method against PARIS*-m and DTA. As shown, our approach significantly outperforms both DTA and PARIS*-m across all multi-part objects in both joint estimation and mesh reconstruction. As shown on Figure 6 as the number of parts increases, baseline methods struggle to accurately recover the articulation model and object geometry. In contrast, our method maintains strong performance regardless of the number of parts. We, again, note that our method is capable of achieving such accurate results only from a casual video input. These results highlight the robustness and efficiency of our approach, not only in terms of data requirements but also in overall performance.

4.5. Ablation Study

We conducted an ablation study on our hypotheses selection. We selected two multi-part objects, each containing

one prismatic joint and one revolute joint. As shown in Table 3, initializing the joint parameters with the hypothesis set $\hat{\mathcal{C}}$ that has the lowest $\text{BIC}(\hat{\mathcal{C}})$ score yields the most accurate results. Selecting the hypothesis set with the second-lowest $\text{BIC}(\mathcal{C})$ score leads to suboptimal outcomes and may even cause the final optimization to diverge.

The last row of Table 3 presents results without the proposed model selection step. In this case, the number of parts is assumed to be known, and the models representing the joints are selected by taking the hypotheses with the largest number of inliers. The table shows that this simple strategy is far from optimal.

5. Conclusion

We introduced a robust and accurate method that can recover the geometry and the joint parameters of an articulated object from a single video. Our method relies on 3D trajectories that we use to generate a set of hypotheses on the joint parameters. We introduced an efficient method to identify the subset of hypotheses that actually explain the object. We use the trajectories not only to find the number of parts and initialize the joint parameters, but also to guide the optimization of the object geometry. Our approach significantly outperforms state of the art on multi-part objects.

References

- [1] Quankai Gao, Qiangeng Xu, Zhe Cao, Ben Mildenhall, Wen-chao Ma, Le Chen, Danhang Tang, and Ulrich Neumann. Gaussianflow: Splatting Gaussian Dynamics for 4D Content Creation. In *arXiv Preprint*, 2024. 3
- [2] Haoran Geng, Ziming Li, Yiran Geng, Jiayi Chen, Hao Dong, and He Wang. Partmanip: Learning Cross-Category Generalizable Part Manipulation Policy from Point Cloud Observations. In *CVPR*, 2023. 2
- [3] Haoran Geng, Helin Xu, Chengyang Zhao, Chao Xu, Li Yi, Siyuan Huang, and He Wang. Gapartnet: Cross-Category Domain-Generalizable Object Perception and Manipulation via Generalizable and Actionable Parts. In *CVPR*, 2023. 2
- [4] Richard I. Hartley and Andrew Zisserman. Multiple Views Geometry in Computer Vision. *Cambridge University Press*, 2000. 5
- [5] Nick Heppert, Muhammad Zubair Irshad, Sergey Zakharov, Katherine Liu, Rares Andrei Ambrus, Jeannette Bohg, Abhinav Valada, and Thomas Kollar. Carto: Category and Joint Agnostic Reconstruction of Articulated Objects. In *CVPR*, 2023. 1, 2
- [6] Hanxiao Jiang, Yongsan Mao, Manolis Savva, and Angel X. Chang. Opd: Single-View 3D Openable Part Detection. In *ECCV*, 2022.
- [7] Zhenyu Jiang, Cheng-Chun Hsu, and Yuke Zhu. Ditto: Building Digital Twins of Articulated Objects from Interaction. In *CVPR*, 2022. 1, 2, 3, 6
- [8] Nikita Karaev, Iurii Makarov, Jianyuan Wang, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. Co-Tracker3: Simpler and Better Point Tracking by Pseudo-Labeling Real Videos. In *arXiv Preprint*, 2024. 2, 4
- [9] Kai Katsumata, Duc Minh Vo, and Hideki Nakayama. An Efficient 3D Gaussian Representation for Monocular/multi-View Dynamic Scenes. In *arXiv Preprint*, 2023. 3
- [10] Yuki Kawana and Tatsuya Harada. Detection Based Part-Level Articulated Object Reconstruction from Single RGBD Image. In *NeurIPS*, 2023. 2
- [11] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM SIGGRAPH*, 2023. 7
- [12] Justin Kerr, Chung Min Kim, Mingxuan Wu, Brent Yi, Qianqian Wang, Ken Goldberg, and Angjoo Kanazawa. Robot See Robot Do: Imitating Articulated Object Manipulation with Monocular 4D Reconstruction. In *CoRL*, 2024. 1, 2
- [13] Agelos Kratimenos, Jiahui Lei, and Kostas Daniilidis. Dynmf: Neural Motion Factorization for Real-Time Dynamic View Synthesis with 3D Gaussian Splatting. In *arXiv Preprint*, 2023. 3
- [14] Long Le, Jason Xie, William Liang, Hung-Ju Wang, Yue Yang, Yecheng Jason Ma, Kyle Vedder, Arjun Krishna, Dinesh Jayaraman, and Eric Eaton. Articulate-anything: Automatic modeling of articulated objects via a vision-language foundation model. In *ICLR*, 2024. 3, 6
- [15] Xiaolong Li, He Wang, Li Yi, Leonidas J. Guibas, A. Lynn Abbott, and Shuran Song. Category-Level Articulated Object Pose Estimation. In *CVPR*, 2020. 1, 2
- [16] Zhan Li, Zhang Chen, Zhong Li, and Yi Xu. Spacetime Gaussian Feature Splatting for Real-Time Dynamic View Synthesis. In *CVPR*, 2023. 3
- [17] Yiqing Liang, Numair Khan, Zhengqin Li, Thu Nguyen-Phuoc, Douglas Lanman, James Tompkin, and Lei Xiao. GauFR: Gaussian Deformation Fields for Real-Time Dynamic Novel View Synthesis. In *arXiv Preprint*, 2023. 3
- [18] Jiayi Liu, Ali Mahdavi-Amiri, and Manolis Savva. Paris: Part-Level Reconstruction and Motion Analysis for Articulated Objects. In *ICCV*, 2023. 1, 2, 3, 6, 7
- [19] Jonathon Luiten, Georgios Kopanas, Bastian Leibe, and Deva Ramanan. Dynamic 3D Gaussians: Tracking by Persistent Dynamic View Synthesis. *3DV*, 2023. 3
- [20] Kaichun Mo, Shilin Zhu, Angel X. Chang, Li Yi, Subarna Tripathi, Leonidas J. Guibas, and Hao Su. PartNet: A Large-Scale Benchmark for Fine-Grained and Hierarchical Part-Level 3D Object Understanding. In *CVPR*, 2019. 6
- [21] Jiteng Mu, Weichao Qiu, Adam Kortylewski, Alan Yuille, Nuno Vasconcelos, and Xiaolong Wang. A-SDF: Learning Disentangled Signed Distance Functions for Articulated Shape Representation. In *ICCV*, 2021. 1, 2
- [22] Atsuhiko Noguchi, Umar Iqbal, Jonathan Tremblay, Tatsuya Harada, and Orazio Gallo. Watch It Move: Unsupervised Discovery of 3D Joints for Re-Posing of Articulated Objects. In *CVPR*, 2022. 3
- [23] Kei Ota, Hsiao-Yu Tung, Kevin A. Smith, Anoop Cherian, Tim K. Marks, Alan Sullivan, Asako Kanezaki, and Joshua B. Tenenbaum. h-saur: Hypothesize, simulate, act, update, and repeat for understanding object articulations from interactions. In *International Conference on Robotics and Automation*, 2023. 3
- [24] Jeong Joon Park, Peter Florence, Julian Straub, Richard A. Newcombe, and Steven J. Lovegrove. DeepSDF: Learning Continuous Signed Distance Functions for Shape Representation. In *CVPR*, 2019. 2
- [25] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach Devito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *International Conference on Neural Information Processing*, 2019. 7
- [26] Luigi Piccinelli, Yung-Hsu Yang, Christos Sakaridis, Mattia Segu, Siyuan Li, Luc Van Gool, and Fisher Yu. UniDepth: Universal Monocular Metric Depth Estimation. In *CVPR*, 2024. 2, 4
- [27] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment Anything in Images and Videos. In *arXiv Preprint*, 2024. 7
- [28] Gideon E. Schwarz. Estimating the Dimension of a Model. *Annals of Statistics*, 1978. 5

- [29] Chaoyue Song, Jiacheng Wei, Chuan-Sheng Foo, Guosheng Lin, and Fayao Liu. REACTO: Reconstructing Articulated Objects from a Single Video. In *CVPR*, 2024. [1](#), [2](#), [3](#)
- [30] Xiaohao Sun, Hanxiao Jiang, Manolis Savva, and Angel Xuan Chang. Opdmulti: Openable Part Detection for Multiple Objects. In *arXiv Preprint*, 2023. [2](#)
- [31] He Wang, Srinath Sridhar, Jingwei Huang, Julien Valentin, Shuran Song, and Leonidas J. Guibas. Normalized Object Coordinate Space for Category-Level 6D Object Pose and Size Estimation. In *CVPR*, 2019. [2](#)
- [32] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. Vggt: Visual geometry grounded transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025. [7](#)
- [33] Qianqian Wang, Vickie Ye¹, Hang Gao, Jake Austin, Zhengqi Li, and Angjoo Kanazawa. Shape of Motion: 4D Reconstruction from a Single Video. In *arXiv Preprint*, 2024. [3](#), [5](#)
- [34] Yijia Weng, He Wang, Qiang Zhou, Yuzhe Qin, Yueqi Duan, Qingnan Fan, Baoquan Chen, Hao Su, and Leonidas J. Guibas. CAPTRA: CAtegory-level Pose Tracking for Rigid and Articulated Objects from Point Clouds. In *ICCV*, 2021. [2](#)
- [35] Yijia Weng, Bowen Wen, Jonathan Tremblay, Valts Blukis, Dieter Fox, Leonidas Guibas, and Stan Birchfield. Neural Implicit Representation for Building Digital Twins of Unknown Articulated Objects. In *CVPR*, 2024. [1](#), [2](#), [3](#), [6](#), [7](#)
- [36] Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 4D Gaussian Splatting for Real-Time Dynamic Scene Rendering. In *CVPR*, 2024. [3](#)
- [37] Ziyi Yang, Xinyu Gao, Wen Zhou, Shaohui Jiao, Yuqing Zhang, and Xiaogang Jin. Deformable 3D Gaussians for High-Fidelity Monocular Dynamic Scene Reconstruction. In *CVPR*, 2024. [2](#), [3](#)